

HEAS: Hierarchical Evolutionary Agent Simulation Framework for Cross-Scale Modeling and Multi-Objective Search

Ruiyu Zhang¹, Lin Nie², and Xin Zhao²

¹ Department of Politics and Public Administration, The University of Hong Kong, Hong Kong SAR, China ² Department of Applied Social Sciences, The Hong Kong Polytechnic University, Hong Kong SAR, China ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: ¶

Submitted: 20 June 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

HEAS is a Python framework that connects agent-based simulation, evolutionary search, and scenario-based evaluation in a single reproducible pipeline. It is designed for researchers who study systems where local interactions produce system-level outcomes—ecosystems, organizations, markets, or regulatory environments—and who need to search over candidate strategies and compare them across uncertain scenarios. HEAS combines three modules: a hierarchy runtime for composing simulations from reusable process layers, an evolutionary tuner for single- or multi-objective search backed by DEAP, and a game module for evaluating strategies across scenario ensembles. Its central design principle is the *metric contract*: the same outcome function is shared by optimization, evaluation, and validation, so that different parts of an analysis cannot silently rank strategies by different quantities.

Statement of Need

Many research problems now require simulation plus search. Ecologists compare management rules under uncertain shocks; social scientists test institutional designs in heterogeneous populations; engineers tune systems with competing performance and robustness objectives. Agent-based and individual-based models are now used across disciplines to represent complex systems made up of autonomous entities (Grimm et al., 2006). These studies often need the same sequence of steps: build a model, run a search algorithm, evaluate candidate solutions across scenarios, and validate the final ranking.

Several mature tools cover parts of this workflow. Mesa (Villez & others, 2025) and NetLogo (Wilensky, 1999) provide strong foundations for building and exploring agent-based models, but leave multi-objective search and scenario comparison to project-specific scripts. AgentPy (Foramitti, 2021) adds parameter sweeps and Monte Carlo experiments in Python, but does not connect these to evolutionary optimization or tournament evaluation. DEAP (Fortin et al., 2012) offers a flexible evolutionary computation toolkit, but requires the user to wire up simulation coupling, metric definitions, and scenario handling manually. EMA Workbench (Kwakkel, 2017) focuses on exploratory modeling and robust decision making under deep uncertainty, emphasizing many-model experiments rather than multi-objective search over a single simulation hierarchy. OpenMOLE (Reuillon et al., 2013) provides workflow-based model exploration with high-performance computing support, targeting large-scale parameter space exploration rather than integrated simulation-optimization with shared metric definitions.

In each case, the connection between simulation, search, and evaluation is left to the user. A common pattern is to couple an agent-based model to NSGA-II (Deb et al., 2002), then

re-score the resulting strategies in held-out scenarios. The optimizer, tournament evaluator, and validation code must compute the same outcome, yet this consistency is rarely enforced by the framework. Small differences in aggregation can change which strategy appears best without raising an error. HEAS is designed for researchers who need this end-to-end pipeline to be explicit, reproducible, and reusable across substantive domains.

State of the Field

Several frameworks address overlapping parts of the simulation-optimization pipeline. (?) summarizes how HEAS positions itself relative to existing tools.

Capability	Mesa	NetLogo	AgentPy	DEAP	EMA Workbench	OpenMOLE	HEAS
Agent-based modeling	✓	✓	✓	—	—	—	✓
Multi-objective search	—	—	—	✓	—	—	✓
Scenario analysis	—	—	—	—	✓	—	✓
Shared metric across pipeline	—	—	—	—	—	—	✓
Hierarchical composition	—	—	—	—	—	—	✓
Parallel / HPC	—	—	—	—	—	✓	partial
Python-native	✓	—	✓	✓	✓	—	✓

Table 1: Capability comparison across simulation-optimization frameworks. “Supports” (✓) indicates the framework provides built-in primitives for that capability; “—” indicates no native support; “partial” indicates limited or experimental support. OpenMOLE orchestrates external ABM models (e.g. from NetLogo) but does not provide native agent classes or scheduling primitives. EMA Workbench supports exploratory scenario analysis and scenario discovery, but uses many-model ensembles rather than voting-rule-based tournaments. “Shared metric across pipeline” means the same outcome function is enforced by the framework across optimization, evaluation, and validation. “Hierarchical composition” means simulations can be assembled from layered, reusable process components.

Mesa (Villez & others, 2025) and NetLogo (Wilensky, 1999) are strong choices for building and exploring agent-based models, but they do not provide integrated multi-objective search or scenario-based evaluation. AgentPy (Foramitti, 2021) adds parameter sweeps and Monte Carlo experiments, but does not connect these to evolutionary optimization. DEAP (Fortin et al., 2012) provides modular evolutionary algorithms; HEAS builds on DEAP rather than replacing it. EMA Workbench (Kwakkel, 2017) supports exploratory modeling and robust decision making under deep uncertainty, but frames the workflow around many-model ensembles rather than multi-objective search within a single simulation hierarchy. OpenMOLE (Reuillon et al., 2013) orchestrates model exploration across high-performance computing resources, but does not enforce metric consistency across the optimization and evaluation stages.

68 HEAS contributes the missing integration layer: it connects hierarchical simulation, multi-
69 objective search, scenario comparison, and a shared outcome definition in one framework. This
70 makes HEAS useful to modelers who already know how to write a simulation, but need a more
71 reliable bridge from simulation to search and comparative evaluation.

72 Software Design

73 HEAS has three modules that share a common data flow.

74 **Hierarchy runtime.** A *stream* is a user-defined process that reads from and writes to a shared
75 simulation context. Streams are the basic units of composition: each one encapsulates a piece
76 of domain logic (an ecological process, a firm, a regulator, or an ODE system) while remaining
77 agnostic about how other streams behave. A *layer* groups streams that execute at the same
78 temporal resolution. A *hierarchy* orders layers so that slower processes (e.g. seasonal dynamics)
79 frame faster ones (e.g. daily agent decisions). The following example defines a stream and
80 registers it in a two-layer hierarchy:

```
from heas.hierarchy.base import Stream, Context
from heas.hierarchy.orchestrator import (
    LayerSpec, StreamSpec, make_model_from_spec,
)

class Prey(Stream):
    def __init__(self, name, ctx, growth=0.05, **kw):
        super().__init__(name, ctx)
        self.growth = growth
        self.pop = 100.0

    def step(self):
        self.pop *= 1 + self.growth
        self.ctx.data["prey"] = self.pop

class Predator(Stream):
    def __init__(self, name, ctx, conversion=0.02, **kw):
        super().__init__(name, ctx)
        self.conversion = conversion

    def step(self):
        prey = self.ctx.data.get("prey", 0)
        self.ctx.data["predators"] = prey * self.conversion

spec = [
    LayerSpec(streams=[StreamSpec("prey", Prey, {"growth": 0.05})]),
    LayerSpec(streams=[StreamSpec("pred", Predator, {"conversion": 0.02})]),
]
model_factory = make_model_from_spec(spec, seed=42)
```

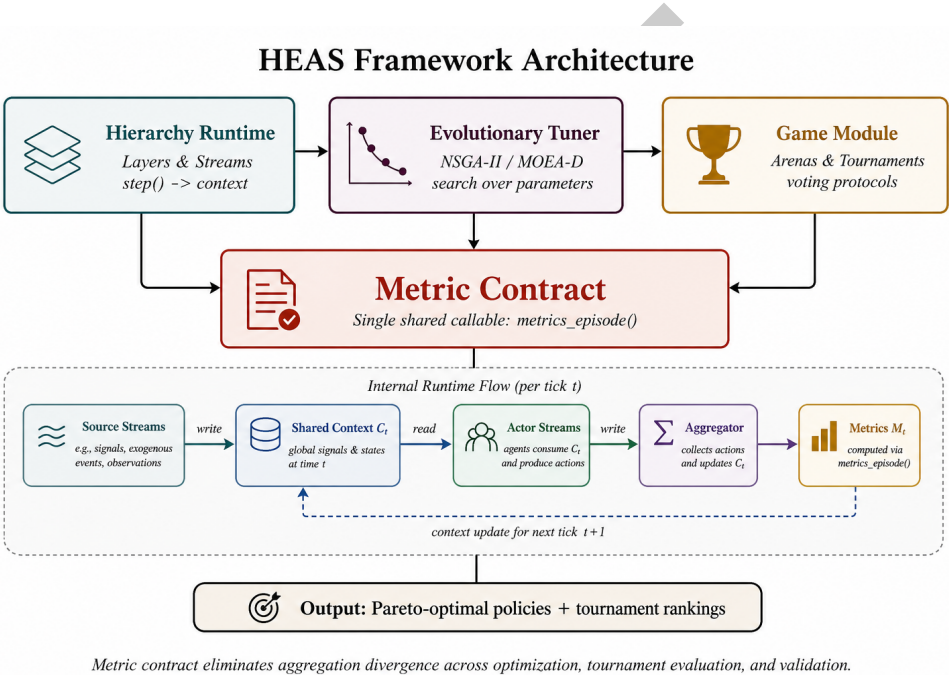
81 The hierarchy runtime executes layers in order and passes the shared context downstream, so
82 the same stream can be reused in different hierarchies without modification.

83 **Evolutionary tuner.** The tuner connects DEAP-backed single- and multi-objective search (NSGA-
84 II (Deb et al., 2002), MOEA/D (Q. Zhang & Li, 2007)) to the hierarchy. A *gene schema*
85 maps candidate parameter vectors to stream parameters, and a single `metrics_episode()`
86 callable evaluates each episode. Deterministic seeding and parallel episode execution ensure
87 reproducibility.

88 **Game module.** The game module evaluates candidate strategies across scenario ensembles and

89 aggregates results using configurable voting rules (argmax, majority, Borda count, Copeland
90 pairwise majority). It receives the same `metrics_episode()` used by the tuner, so rankings
91 are guaranteed to be computed from the same outcome definition.

92 The main design trade-off is that HEAS keeps domain behavior in user-defined streams rather
93 than imposing a single scientific model type. This is less prescriptive than a domain-specific
94 simulator, but it lets the same workflow serve ecology, organizational analysis, policy design,
95 and mathematical systems modeling. The metric contract is the corresponding constraint:
96 users define the outcome once, and the same definition is passed through search, tournament
97 evaluation, and validation.



Metric contract eliminates aggregation divergence across optimization, tournament evaluation, and validation.

Figure 1: HEAS Framework Architecture

98 *Figure 1: HEAS three-module architecture. The Hierarchy Runtime composes simulations*
99 *from layers and streams, the Evolutionary Tuner performs multi-objective search (NSGA-*
100 *II/MOEA/D), and the Game Module evaluates policies across scenario ensembles. The Metric*
101 *Contract ensures all three modules compute the same outcome metric through a single shared*
102 *callable, eliminating aggregation divergence.*

103 Use Cases

104 HEAS has been applied to three reference studies that exercise different scientific logics while
105 keeping the framework pipeline fixed. In each case, only the stream factories, gene schemas,
106 scenarios, and episode metrics change; the hierarchy, search, tournament, and metric-contract
107 interfaces remain the same. These studies demonstrate that the same framework surface
108 supports both evolutionary search and held-out scenario evaluation without rewiring, which is
109 the main portability claim of the software. The enterprise regulatory design case is currently
110 documented in the literature (R. Zhang et al., 2026b), while the ecological and Wolf-Sheep
111 configurations serve as reference demonstrations of the same framework interface across
112 different model families.

113 **Ecological population management** assembles a predator-prey arena with five streams, a 2-gene

114 policy (risk, dispersal), and fragmentation $\times$ shock scenarios. The same model-construction
115 and evaluation surface supported both evolutionary search and a held-out 64-scenario robustness
116 check without modification.

117 **Enterprise regulatory design** reuses the same framework contracts for a four-layer regulatory
118 arena with a 4-gene policy (tax_rate, audit_intensity, subsidy, penalty_rate). The tuner
119 ran NSGA-II over a larger scenario ensemble. The domain logic changes substantially, but the
120 search and tournament wiring do not.

121 **Wolf-Sheep ODE** keeps the framework interfaces fixed while swapping the underlying model
122 family from an agent simulation to a mean-field Lotka-Volterra system. This case is the
123 clearest portability test: the 4-layer arena required no framework-side coupling beyond a new
124 metrics_episode() implementation.

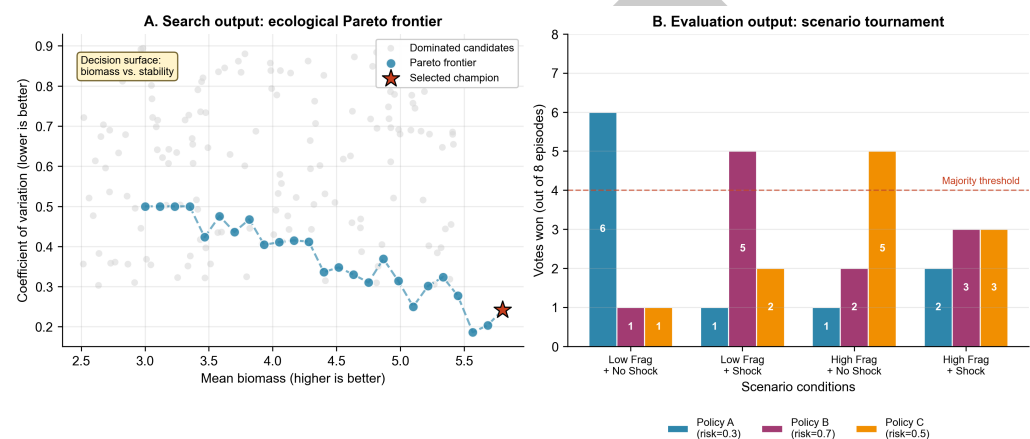


Figure 2: Ecological Arena Results

125 *Figure 2: Example outputs from the ecological arena case study. Left: Pareto front showing*
126 *the trade-off between mean biomass (higher is better) and coefficient of variation (lower is*
127 *better). The star marks the champion policy selected by HEAS. Right: Tournament voting*
128 *summary showing how different policies perform across scenario conditions, with majority*
129 *threshold indicated.*

130 Availability and Reproducibility

131 HEAS includes a command-line interface for batch runs and a browser-based playground at
132 <https://ryzhanghason.github.io/heas/> for configuring simulations, inspecting Pareto fronts,
133 and exporting publication bundles without a backend server. The repository includes a
134 CONTRIBUTING.md guide, a CODE_OF_CONDUCT.md, release metadata, automated tests, and a
135 versioned Zenodo archive (R. Zhang et al., 2026a).

136 Research Impact Statement

137 HEAS has already supported policy analysis, including policy diffusion settings, and
138 organizational strategy optimization. Its research value is to make these workflows reproducible
139 within one framework by keeping simulation, search, and scenario evaluation aligned through a
140 shared metric contract. In these settings, HEAS has been useful for comparing candidate
141 strategies across heterogeneous scenarios while preserving one explicit outcome definition from
142 optimization through final evaluation (R. Zhang et al., 2026b).

143 Acknowledgements

144 The authors acknowledge the open-source simulation and optimization communities whose
145 tools and documentation informed HEAS, especially Mesa, NetLogo, AgentPy, DEAP, EMA
146 Workbench, and OpenMOLE.

147 AI Usage Disclosure

148 The authors used AI-assisted tools in limited support roles. The authors take full responsibility
149 for all submitted work.

150 References

- 151 Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002). A fast and elitist multiobjective
152 genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2),
153 182–197. <https://doi.org/10.1109/4235.996017>
- 154 Foramitti, J. (2021). AgentPy: A package for agent-based modeling in Python. *Journal of*
155 *Open Source Software*, 6(62), 3065. <https://doi.org/10.21105/joss.03065>
- 156 Fortin, F.-A., De Rainville, F.-M., Gardner, M.-A., Parizeau, M., & Gagné, C. (2012). DEAP:
157 Evolutionary algorithms made easy. *Journal of Machine Learning Research*, 13, 2171–2175.
158 <https://dl.acm.org/doi/10.5555/2503308.2503311>
- 159 Grimm, V., Berger, U., Bastiansen, F., Eliassen, S., Ginot, V., Giske, J., Goss-Custard, J.,
160 Grand, T., Heinz, S. K., Huse, G., Huth, A., Jepsen, J. U., Jørgensen, C., Mooij, W. M.,
161 Müller, B., Pe'er, G., Piou, C., Railsback, S. F., Robbins, A. M., ... DeAngelis, D. L. (2006).
162 A standard protocol for describing individual-based and agent-based models. *Ecological*
163 *Modelling*, 198(1), 115–126. <https://doi.org/10.1016/j.ecolmodel.2006.04.023>
- 164 Kwakkel, J. H. (2017). The exploratory modeling workbench: An open source toolkit for
165 exploratory modeling, scenario discovery, and (multi-objective) robust decision making.
166 *Environmental Modelling & Software*, 96, 239–250. <https://doi.org/10.1016/j.envsoft.2017.06.054>
- 168 Reuillon, R., Leclaire, M., & Rey-Coyrehourcq, S. (2013). OpenMOLE, a workflow engine
169 specifically tailored for the distributed exploration of simulation models. *Future Generation*
170 *Computer Systems*, 29(8), 1981–1990. <https://doi.org/10.1016/j.future.2013.05.003>
- 171 Villez, K., & others. (2025). Mesa: An agent-based modeling framework for python. *Journal*
172 *of Open Source Software*, 10(108), 7668. <https://doi.org/10.21105/joss.07668>
- 173 Wilensky, U. (1999). *NetLogo*. Center for Connected Learning and Computer-Based Modeling,
174 Northwestern University, Evanston, IL. <http://ccl.northwestern.edu/netlogo/>
- 175 Zhang, Q., & Li, H. (2007). MOEA/D: A multiobjective evolutionary algorithm based
176 on decomposition. *IEEE Transactions on Evolutionary Computation*, 11(6), 712–731.
177 <https://doi.org/10.1109/TEVC.2007.892759>
- 178 Zhang, R., Nie, L., & Zhao, X. (2026a). *HEAS: Hierarchical evolutionary agent simulation*
179 *framework, version 1.0.2*. Zenodo. <https://doi.org/10.5281/zenodo.20696882>
- 180 Zhang, R., Nie, L., & Zhao, X. (2026b). *Metric aggregation divergence: A hidden validity*
181 *threat in agent-based policy optimization and a contractual remedy*. <https://doi.org/10.48550/arXiv.2606.29038>
- 182